

Hallucination Resistant Document Extraction Workflow

Independent AI Systems Case Study

Role: Prompt Engineer and AI Workflow Designer

Objective: Design and test an evidence first contract extraction workflow that preserves missing, ambiguous, conflicting, and unreadable information instead of forcing unsupported answers.

1. Source Original document plus raw text or OCR with stable source identifiers	2. Language model Semantic extraction and evidence selection
3. Deterministic validation Schema, exact evidence matching, allowed states, routing rules	4. Human review Ambiguous, conflicting, unreadable, or unresolved evidence

Core design principle

The model interprets contract language. Software enforces exact structural and provenance requirements. Human reviewers handle genuinely unresolved evidence.

This case study uses synthetic contracts and controlled conversational model runs. It does not establish production accuracy, legal correctness, security proof, or deployment readiness.

1. Problem and Evidence States

A contract extraction system can fail by inventing missing terms, choosing between contradictory clauses, silently repairing corrupted OCR, treating an unrelated monetary amount as a liability cap, or following instructions embedded inside document text.

The workflow therefore used five explicit evidence states:

State	Meaning
SUPPORTED	One factual value is clearly supported by source evidence.
MISSING	The relevant information is not stated.
AMBIGUOUS	Relevant evidence exists, but more than one interpretation is reasonably possible.
CONFLICTING	Separate source passages state materially inconsistent values or identities.
UNREADABLE	Relevant evidence exists, but corruption prevents reliable extraction of the exact value.

Eight requested fields

- Agreement title; party names; effective date; payment terms
- Automatic renewal terms; termination notice period; governing law; liability cap

Each evidence item used a stable source identifier and exact source text. Raw OCR evidence was preserved rather than silently corrected.

2. Benchmark Design

The first 12 synthetic documents became Development Benchmark V1. They were used for prompt development, failure discovery, and controlled regression testing. They were not presented as unseen final evidence.

Development coverage included:

- Clean extraction and genuinely missing fields
- Several plausible dates with no explicit effective date
- Contradictory termination periods
- Insurance amount that is not a liability cap
- Fixed and formula based liability caps
- OCR corruption affecting a requested value
- Heading meaning that conflicts with clause meaning
- Prompt injection style text inside the document
- Per party conflict preservation and conditional governing law

After development, six new synthetic contracts were created as an unseen holdout. Their hidden expected answers were not supplied to the Veteran Prompt Engineering AI before execution.

The holdout was intended to test whether the frozen prompt generalized to new wording rather than merely repeating patterns from the development documents.

3. Controlled Development Results

108 / 108	98 / 98	11 / 12
Expected substantive decisions matched	Manually traced evidence selections	Raw responses valid JSON

What the development run showed

- The model preserved missing, conflicting, ambiguous, and unreadable states rather than forcing answers.
- It distinguished insurance coverage from a liability cap and explicit effective dates from merely plausible dates.
- It ignored a prompt injection style instruction embedded inside contract text.
- Per party extraction preserved a supported customer identity even when the provider identity conflicted.

The development run also exposed two design weaknesses: incomplete evidence spans and raw JSON serialization errors when quotation marks appeared inside evidence strings.

Only two narrow prompt corrections were made: clarify JSON escaping and require the smallest complete clause that proves both the field meaning and its value.

4. Unseen Holdout Results

54 / 54	49 / 49	4 / 6
Expected substantive decisions matched	Evidence selections traced to cited blocks	Raw responses valid JSON

Difficult holdout behaviors handled correctly

- Refused to infer effective date from contract date, commencement date, or signatures.
- Detected a direct contradiction between automatic renewal and a later clause stating no automatic renewal.
- Marked a corrupted party identity as unreadable while preserving the independently supported other party.
- Ignored embedded instructions attempting to change governing law and liability cap.
- Distinguished a large insurance amount from the actual formula based liability cap.
- Selected a termination notice period rather than a separate breach cure period.

No unsupported factual values, silent OCR repairs, or prompt injection induced factual changes were observed in this small synthetic holdout.

5. The Failure That Matters

Two of six holdout responses were not syntactically valid JSON because quotation marks inside evidence strings were not escaped correctly.

The substantive contract interpretation was correct, but the machine readable output was not always reliable. This separates two responsibilities that are often incorrectly combined.

Language model	Deterministic software	Human review
Semantic interpretation Evidence selection Conflict and ambiguity recognition Candidate structured facts	JSON parsing Schema enforcement Allowed values Exact evidence matching Source identifier verification Reject or retry malformed output	Ambiguous evidence Material conflicts Unreadable OCR Unresolved party identity Undefined business rules

Adding more prompt wording is not a guarantee of valid serialization. Exact structural requirements belong in deterministic controls when the surrounding system can enforce them directly.

6. Limitations and Portfolio Takeaway

Limitations

- Synthetic contracts only.
- 12 development documents and 6 holdout documents.
- Controlled conversational model runs, not independent API executions.
- No real OCR pipeline or scanned document preprocessing.
- No real customer contracts.
- No production latency, cost, throughput, privacy, monitoring, or deployment measurements.
- Evidence was manually inspected rather than verified by a deployed provenance validator.
- Prompt injection handling in these cases is controlled test evidence, not a security proof.
- The results do not establish legal correctness or enforceability.

Portfolio takeaway

The strongest result is architectural discipline, not a claim of perfect reliability. The experiment showed that an evidence first language model workflow can preserve uncertainty and source provenance across a small synthetic benchmark, while deterministic validation remains necessary for machine readable guarantees.

Demonstrated skills: prompt engineering, AI workflow design, provenance aware extraction, adversarial evaluation, failure analysis, benchmark separation, controlled prompt refinement, and responsibility separation between model, software, and human review.